ПРОГРАММИРОВАНИЕ И ОБРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

LINQ и async/await

Лекция 5: спрашиваем данные у коллекции и ждём, не замораживая окно

Практическая лекция: слайды + живой код (LinqDemo, AsyncDemo)

Фоменко М.Ю.

МАРШРУТ КУРСА

Где мы находимся

Было

Лекция 3: делегаты, Func<T>, лямбды. async/await — только на слайдах.

Лекция 4: тесты.

Сегодня

LINQ: лямбды в ежедневной работе с коллекциями.

async/await вживую: почему это не поток.

Применяем

Списки врагов и журнал боёв в кликере, загрузка иконок в редакторе (ЛР1).

Сортировка и фильтры в курсовой.

Обе темы — прямое продолжение делегатов: LINQ принимает Func, а await принимает Task.

КАРТА ЛЕКЦИИ

Четыре блока и примерное время

1. LINQ: основы

~30 мин

Что это, как устроено, основные операторы, два синтаксиса, отложенное выполнение

2. LINQ в деле

~10 мин

Шанс появления врага, журнал боёв, планировщик и тренажёр слов из курсовой

3. async вживую

~30 мин

Поток или ожидание: восемь опытов в AsyncDemo с номерами потоков

4. async в деле

~15 мин

Иконки ЛР1: прогресс, отмена, WhenAll. Грабли. Раунд кликера на await

Ещё ~5 минут остаётся на вопросы. Если не успеваем — блок 2 можно сократить.

1

LINQ

Спрашиваем у коллекции, что нам нужно, а не перебираем её вручную

ПРОБЛЕМА

Циклы на каждый чих

Задача: имена врагов с HP > 100, богатые первыми. Пишем так, как умеем.

```
var strong = new List<CEnemyTemplate>();
foreach (var e in enemies)
    if (e.BaseHealth > 100)
        strong.Add(e);                                  // 1. отобрали

strong.Sort((a, b) => b.BaseGold.CompareTo(a.BaseGold)); // 2. отсортировали

var names = new List<string>();
foreach (var e in strong)
    names.Add(e.Name);                                  // 3. вытащили имена
```

Три временных списка и два цикла ради одной фразы. Завтра нужен другой фильтр — пишем всё заново.

ДО / ПОСЛЕ

Та же задача на LINQ

Было: как делать

```
var strong = new List<CEnemyTemplate>();
foreach (var e in enemies)
    if (e.BaseHealth > 100)
        strong.Add(e);
strong.Sort((a, b) =>
    b.BaseGold.CompareTo(a.BaseGold));
var names = new List<string>();
foreach (var e in strong)
    names.Add(e.Name);
```

Стало: что получить

```
var names = enemies
    .Where(e => e.BaseHealth > 100)
    .OrderByDescending(e => e.BaseGold)
    .Select(e => e.Name)
    .ToList();
```

Результат одинаковый. Но справа видно намерение, а не механику.

ОПРЕДЕЛЕНИЕ

LINQ

LINQ (Language Integrated Query) — набор методов расширения для любой коллекции IEnumerable<T>. Вы описываете, что хотите получить, а перебор пишет библиотека.

1 Любая коллекция

List, массив, Dictionary, строка, результат File.ReadLines — всё, что можно перебрать в foreach.

2 Лямбды на входе

Where принимает Func<T, bool>, Select — Func<T, TResult>. Это делегаты из лекции 3.

3 Цепочки

Каждый метод возвращает новую последовательность, поэтому вызовы ставятся друг за другом.

Подключение: using System.Linq; — в современных проектах уже подключён неявно (ImplicitUsings).

КАК ЭТО УСТРОЕНО

Никакой магии: напишем свой Where

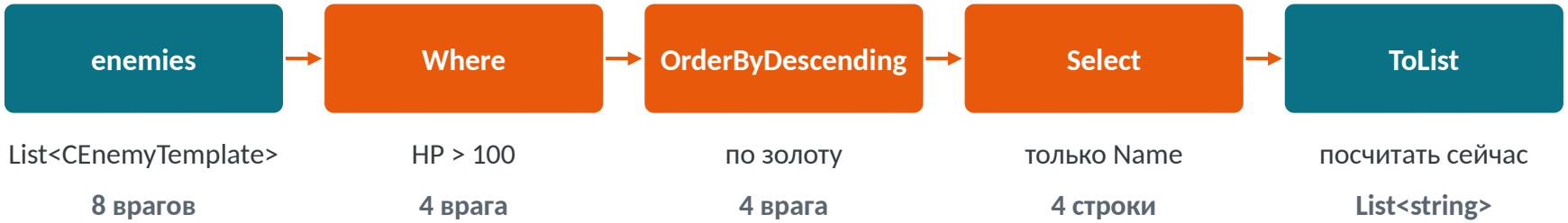
```
public static class MyLinq
{
    public static IEnumerable<T> MyWhere<T>(
        this IEnumerable<T> source,        // this → вызов «через точку»
        Func<T, bool> predicate)           // делегат из лекции 3
    {
        foreach (T item in source)
            if (predicate(item))
                yield return item;         // отдаём по одному, без списка
    }
}

var cheap = enemies.MyWhere(e => e.BaseGold < 10);  // работает как Where
```

Метод расширения + делегат + yield return. Настоящий Where устроен так же, только аккуратнее.

КОНВЕЙЕР

Цепочка — это конвейер



Тип меняется по ходу

IEnumerable<CEnemyTemplate> → … → IEnumerable<string> после Select

Порядок важен

Сначала Where, потом сортировка: сортировать 4 элемента дешевле, чем 8

Читается сверху вниз и слева направо, как обычный текст.

ШПАРГАЛКА

Операторы, которых хватит на 90% задач

Задача	Методы	Вопрос к коллекции
Фильтрация	Where, OfType<T>	Какие из них подходят?
Проекция	Select	Что из каждого взять?
Сортировка	OrderBy, ThenBy, ...Descending	В каком порядке?
Один элемент	First, FirstOrDefault, MaxBy, MinBy	Какой именно?
Агрегаты	Count, Sum, Average, Min, Max	Сколько? Сколько всего?
Да / нет	Any, All, Contains	Есть ли? У всех ли?
Группировка	GroupBy, ToDictionary	Разложить по ключу
Срезы	Take, Skip, Distinct	Первые N? Без повторов?
Выполнить сейчас	ToList, ToArray	Зафиксировать результат

КОД

Where, Select, OrderBy

```
// Where — фильтр
var cheap = enemies.Where(e => e.BaseGold < 10);

// Select — проекция: из врага делаем что угодно, хоть анонимный тип
var cards = enemies.Select(e => new { e.Name,
                                      GoldPerHp = (double)e.BaseGold / e.BaseHealth });

// сортировка по двум ключам: сначала тип, внутри типа — по убыванию HP
var sorted = enemies.OrderBy(e => e.Kind).ThenByDescending(e => e.BaseHealth);

// «вторая страница по 3» и список типов без повторов
var page2 = enemies.OrderBy(e => e.Name).Skip(3).Take(3);
var kinds = enemies.Select(e => e.Kind).Distinct();
```

Повторный OrderBy сбрасывает предыдущую сортировку. Второй ключ задают через ThenBy.

КОД

Один элемент и агрегаты

```
// вместо GetEnemyByName из ЛР1
var goblin = enemies.FirstOrDefault(e => e.Name == "Гоблин");  // нашли
var ghost  = enemies.FirstOrDefault(e => e.Name == "Призрак"); // null, без падения
enemies.First(e => e.Name == "Призрак");    // InvalidOperationException!

int    strongCount = enemies.Count(e => e.BaseHealth > 100);
int    totalGold   = enemies.Sum(e => e.BaseGold);
double avgHp       = enemies.Average(e => e.BaseHealth);
int    maxHp       = enemies.Max(e => e.BaseHealth);     // 2000 — только число
var    boss        = enemies.MaxBy(e => e.BaseHealth);   // Дракон — сам объект

bool anyArmored = enemies.Any(e => e is CArmoredEnemyTemplate);
bool allIcons   = enemies.All(e => !string.IsNullOrEmpty(e.IconPath));
```

Any() вместо Count() > 0: Any останавливается на первом найденном, Count перебирает всё.

СРАВНЕНИЕ

Два синтаксиса — один результат

Синтаксис методов

```
var m = enemies
    .Where(e => e.SpawnChance >= 10)
    .OrderBy(e => e.BaseHealth)
    .Select(e => e.Name);
```

Синтаксис запросов

```
var q = from e in enemies
        where e.SpawnChance >= 10
        orderby e.BaseHealth
        select e.Name;
```

```
var q2 = from e in enemies
         let ratio = (double)e.BaseGold / e.BaseHealth   // промежуточная переменная
         where ratio > 0.18
         orderby ratio descending
         select new { e.Name, ratio };
```

Компилятор превращает query-синтаксис в те же Where / OrderBy / Select. Выбирайте то, что легче читать.

КОД · ЛР4

Список разных врагов: OfType

```
List<CEnemyTemplate> enemies = LoadFromJson();   // обычные, бронированные, лекари

// Where оставляет базовый тип — до Armor не добраться без приведения
var armored1 = enemies.Where(e => e is CArmoredEnemyTemplate);

// OfType фильтрует И приводит: дальше работаем с CArmoredEnemyTemplate
foreach (var a in enemies.OfType<CArmoredEnemyTemplate>())
    Console.WriteLine($"{a.Name}: броня {a.Armor}");

double avgArmor = enemies.OfType<CArmoredEnemyTemplate>().Average(a => a.Armor);
```

Полиморфный список из ЛР4 + OfType = выборка по типу без if (e is …) и ручных приведений.

КОД

GroupBy: разложить по ключу

```
foreach (var g in enemies.GroupBy(e => e.Kind))
{
    // g.Key — значение ключа, сам g — коллекция врагов
    Console.WriteLine($"{g.Key}: {g.Count()} шт., " +
        $"сред. HP {g.Average(e => e.BaseHealth):F0}");
}

var byName = enemies.ToDictionary(e => e.Name);
var golem = byName["Голем"];      // поиск без перебора
```

Вывод

```
Обычный: 4 шт., сред. HP 516
Броня:   2 шт., сред. HP 310
Лекарь:  2 шт., сред. HP 180
```

Одинаковые имена — ToDictionary бросит ArgumentException. Ключ обязан быть уникальным.

ПОДВОДНЫЙ КАМЕНЬ

Отложенное выполнение

```
var strong = enemies.Where(e =>
{
    Console.WriteLine($"проверяю {e.Name}");
    return e.BaseHealth > 100;
});
Console.WriteLine("Запрос создан"); // проверок нет!

enemies.Add(kraken);           // добавили ПОСЛЕ

foreach (var e in strong) ...  // проверки идут тут
int n = strong.Count();        // и снова, с нуля
```

Консоль

```
Запрос создан
проверяю Слизень
проверяю Крыса
…
проверяю Кракен
-> Кракен
проверяю Слизень  ← снова
```

Запрос LINQ — рецепт, а не готовое блюдо. Он выполняется при каждом переборе.

КОГДА ЧТО

Ленивые и немедленные методы

Ленивые: строят рецепт

Where, Select, OrderBy, ThenBy

GroupBy, Skip, Take, Distinct, OfType

Ничего не делают, пока результат не перебрали

Немедленные: готовят

foreach по результату

ToList, ToArray, ToDictionary

Count, Sum, Max, First, Any, All

Нужно один раз зафиксировать результат — добавьте .ToList() в конце.

listBox.ItemsSource = query.ToList(); — интерфейс получает готовый список, а не рецепт.

ЧТОБЫ НЕ НАСТУПИТЬ

Три грабли LINQ

First на пустом	Нет подходящего — InvalidOperationException. Если «не нашли» — нормальная ситуация, берите FirstOrDefault и проверяйте на null.
Повторный перебор	Count(), потом foreach, потом Any() — три прохода с нуля. Дорогой запрос (файлы, расчёт) — сначала ToList().
Изменение во время перебора	foreach по запросу к enemies + enemies.Remove(e) внутри → «Collection was modified». Перебирайте копию: query.ToList().

2

LINQ в деле

Кликер из лабораторных и ваша курсовая

Шанс появления следующего врага

```
double total = enemies.Sum(e => e.SpawnChance);           // сумма весов

var normalized = enemies                                    // доли, в сумме 1
    .Select(e => new { e.Name, P = e.SpawnChance / total })
    .OrderByDescending(x => x.P);

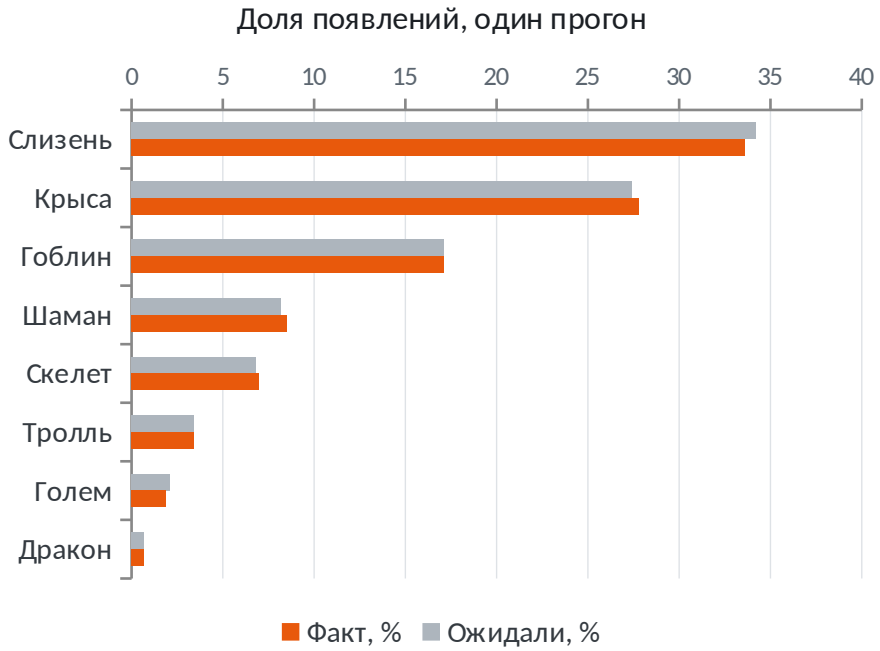
CEnemyTemplate Spawn()
{
    double roll = Random.Shared.NextDouble() * total;       // бросили кубик
    double acc = 0;
    return enemies.First(e => (acc += e.SpawnChance) >= roll); // идём по накопленной сумме
}
```

Нормализация из ЛР2 — две строки вместо двух циклов.

Честный ли генератор? 10 000 спавнов

```
var stats = Enumerable.Range(0, 10_000)
    .Select(_ => Spawn().Name)
    .GroupBy(name => name)
    .Select(g => new { g.Key,
        Share = g.Count() / 10_000.0 });
```

Факт и ожидание совпадают с точностью до долей процента.



GroupBy + Join: статистика сессии

```
var perEnemy = kills
    .GroupBy(k => k.EnemyName)
    .Select(g => new {
        Enemy = g.Key,
        Times = g.Count(),
        Gold = g.Sum(k => k.GoldReceived),
        AvgClicks = g.Average(k => k.Clicks) })
    .OrderByDescending(x => x.Gold);

var goldByKind = kills
    .Join(enemies, k => k.EnemyName, e => e.Name,        // ключ слева, ключ справа
          (k, e) => new { e.Kind, k.GoldReceived })      // что получить из пары
    .GroupBy(x => x.Kind)
    .Select(g => $"{g.Key}: {g.Sum(x => x.GoldReceived)} золота");
```

Готовая статистика для окна «Итоги раунда» — без единого цикла.

КУРСОВАЯ · ВАРИАНТ 2

Планировщик задач

```
// сортировка: ближайший срок первым, при равенстве — важнее выше
var sorted = tasks.OrderBy(t => t.Deadline).ThenByDescending(t => t.Priority);

// фильтр из ComboBox: null = «Все», иначе выбранный статус
TodoStatus? selected = (TodoStatus?)statusBox.SelectedValue;
var filtered = sorted.Where(t => selected == null || t.Status == selected);

// просроченные и невыполненные — подсветить красным
var overdue = tasks.Where(t => t.Deadline < DateTime.Today && t.Status != TodoStatus.Done);

tasksList.ItemsSource = filtered.ToList();
```

Весь функционал «сортировка + фильтр» из задания курсовой — три выражения LINQ.

КУРСОВАЯ · ВАРИАНТ 1

Тренажёр иностранных слов

```
var categories = words.Select(w => w.Category).Distinct().Order();   // в ComboBox

var accuracy = words.GroupBy(w => w.Category)
    .Select(g => new {
        Category = g.Key,
        Percent  = (double)g.Sum(w => w.Correct) / g.Sum(w => w.Correct + w.Wrong) });

var hardest = words.Where(w => w.Correct + w.Wrong > 0)
    .OrderByDescending(w => (double)w.Wrong / (w.Correct + w.Wrong))
    .Take(3);

var next = words.Where(w => w.Category == selectedCategory)
    .OrderBy(_ => Random.Shared.Next())                    // перемешали
    .First();
```

Деление на ноль: категория, где ещё не тренировались. В LinqDemo это учтено, проверьте у себя.

ИТОГ ЧАСТИ 1–2

LINQ: что забираем с собой

1. LINQ — методы расширения IEnumerable<T>, которые принимают лямбды. Никакой магии, только делегаты.
2. Where / Select / OrderBy / GroupBy + агрегаты закрывают почти все задачи лабораторных и курсовой.
3. Запрос ленивый: выполняется при переборе и при каждом переборе заново. Нужно зафиксировать — ToList().
4. FirstOrDefault вместо First, Any вместо Count() > 0, ToList перед изменением исходного списка.

Но сначала данные нужно загрузить. Файлы, сеть, база — и окно не должно при этом зависать.

3

async/await вживую

Поток — это исполнитель. await — это ожидание без исполнителя

НАПОМИНАНИЕ · ЛЕКЦИЯ 3

Что мы уже знаем на словах

1. **await не создаёт поток.** Он говорит: «эта работа займёт время, верни поток тому, кто его ждёт».

2. **async-метод возвращает Task.** Task — обещание результата, Task<T> — обещание значения типа T.

3. **Код после await продолжится сам,** а в WPF вернётся в поток интерфейса.

Сегодня проверяем каждое утверждение руками: смотрим на номера потоков в логе.

ГЛАВНАЯ ИДЕЯ

Повар и духовка

Thread.Sleep / .Result

Повар стоит у духовки и смотрит в стекло. Зал без обслуживания: окно «Не отвечает».

new Thread / Task.Run

Позвали второго повара. Работает, но второй повар — ресурс: память, переключения.

await Task.Delay / ReadAsync

Поставил в духовку, поставил таймер и пошёл в зал. Пока печётся, повара нет вообще.

Поток — исполнитель. await — способ подождать, не занимая исполнителя.

ЛАЙВ · ASYNCDEMO, ВКЛАДКА 1

Восемь опытов

№	Что делаем	Пульс UI	Потоки в логе
1	Thread.Sleep(5000) в обработчике	**замер**	T1
2	async-метод без await внутри, тяжёлый расчёт	**замер**	T1, T1
3	new Thread(…)	**живой**	T1, затем другой
4	Из чужого потока тронуть элемент UI	**живой**	исключение
5	await Task.Run(расчёт)	**живой**	T1 → пул → T1
6	await Task.Delay(3000)	**живой**	T1 → T1
7	1000 × await Task.Delay(2000)	**живой**	потоков ≈ не прибавилось
8	100 × new Thread(Sleep(2000))	**живой**	+100 потоков

ОПЫТ 2

Слово async само ничего не делает

```
private async Task<int> CountPrimesFakeAsync()
{
    Log("внутри «асинхронного» метода");      // [T1]
    return CountPrimes(4_000_000);             // считаем в том же потоке
}                                              // warning CS1998: нет await

private async void FakeAsync_Click(object sender, RoutedEventArgs e)
{
    int result = await CountPrimesFakeAsync();  // окно всё равно висит
    Log($"готово, {result} простых");            // [T1]
}
```

async — это разрешение писать await внутри метода. Поток он не создаёт и работу не ускоряет.

Настоящий второй поток

```
var worker = new Thread(() =>
{
    Log("я в другом потоке");          // [T9]
    int n = CountPrimes(4_000_000);    // окно живое: считает не T1

    PulseText.Text = "ВЗЛОМ";          // InvalidOperationException:
                                       // владелец объекта — другой поток
    Dispatcher.Invoke(() => PulseText.Text = "так можно");
});
worker.IsBackground = true;            // не держит процесс после закрытия окна
worker.Start();
```

Отдельный поток работает, но с ним приходится вручную передавать результат обратно в UI.

ОПЫТЫ 5-6

Task.Run и Task.Delay: найдите разницу

Считаем (CPU)

```
Log("до");                  // [T1]
int n = await Task.Run(() =>
{
    Log("внутри");          // [T6]  пул
    return CountPrimes(4_000_000);
});
Log("после");               // [T1]
```

Ждём (I/O, таймер)

```
Log("до");                  // [T1]
await Task.Delay(3000);
// 3 секунды: никакой поток
// не ждёт, ждёт таймер ОС
Log("после");               // [T1]
```

Task.Run — «позвать повара из пула». await Task.Delay — «поставить таймер и уйти».

После await в WPF мы снова в потоке UI: элементы окна можно трогать без Dispatcher.

ОПЫТЫ 7-8

1000 ожиданий против 100 потоков

≈ +0

потоков на 1000 × await Task.Delay

Все 1000 закончились примерно через 2 с

+100

потоков на 100 × Thread.Sleep

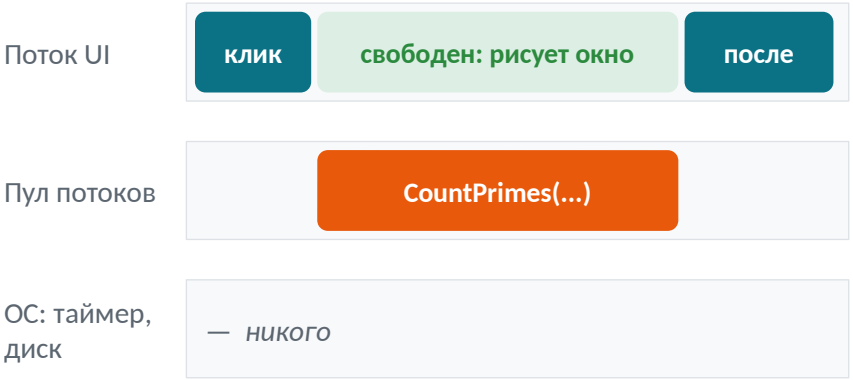
Каждый спящий поток держит около 1 МБ стека

```
Task[] waits = Enumerable.Range(0, 1000)       // LINQ + async
                         .Select(_ => Task.Delay(2000))
                         .ToArray();
await Task.WhenAll(waits);                     // ждём все сразу
```

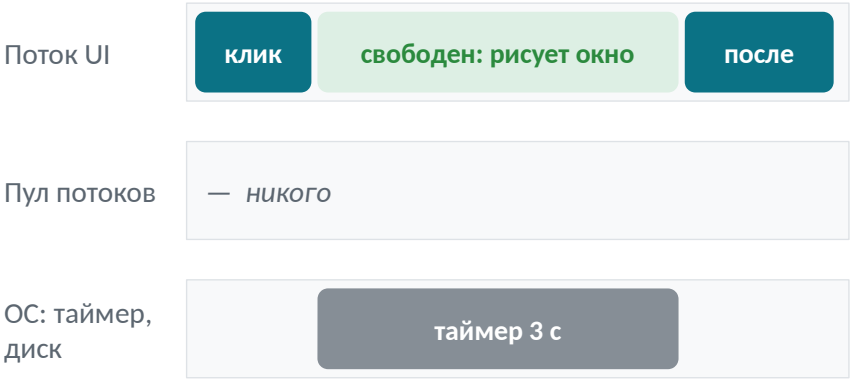
КАК ЭТО ВЫГЛЯДИТ ВО ВРЕМЕНИ

Кто чем занят во время await

`await Task.Run(расчёт)`



`await Task.Delay(3000)`



В обоих случаях поток UI свободен. Разница — кто выполняет работу: поток пула или никто.

КОГДА ЧТО

Ждём, считаем или нужен поток

Ждём ответа

Файл, сеть, база, таймер.

Процессор не нужен.

→ await …Async()

Считаем сами

Расчёт, обработка картинки, разбор большого файла.

→ await Task.Run(…)

Свой поток

Долгоживущая фоновая работа на всё время программы.

→ new Thread — редко

Есть метод с суффиксом Async — используйте его. Нет — заверните тяжёлый расчёт в Task.Run.

4

async в деле

Иконки из ЛР1, отмена, прогресс, грабли и раунд кликера

ПРОБЛЕМА · ЛР1

Редактор врагов зависает на загрузке

```
public BitmapSource LoadOne(string path)
{
    Thread.Sleep(15);                          // «медленный диск»
    byte[] bytes = File.ReadAllBytes(path);    // поток стоит и ждёт диск
    return Decode(bytes);
}

foreach (string file in files)                 // 300 файлов × ~16 мс ≈ 5 с
    _icons.Add(_loader.LoadOne(file));         // окно не рисуется до конца цикла
```

Пять секунд белого окна. Пользователь думает, что программа упала.

КОД

Загрузчик: async, прогресс и отмена

```
public async Task<BitmapSource> LoadOneAsync(string path, CancellationToken ct)
{
    await Task.Delay(15, ct);                          // ждём, поток свободен
    byte[] bytes = await File.ReadAllBytesAsync(path, ct);
    return Decode(bytes);
}

public async Task<int> LoadAllSequentialAsync(IEnumerable<string> files,
    IProgress<BitmapSource>? progress, CancellationToken ct)
{
    int count = 0;
    foreach (string file in files)
    {
        ct.ThrowIfCancellationRequested();             // нажали «Отмена» — выходим
        progress?.Report(await LoadOneAsync(file, ct)); // отдали иконку наружу
        count++;
    }
    return count;
}
```

код

Окно: Progress<T> и CancellationTokenSource

```
_cts = new CancellationTokenSource();
var progress = new Progress<BitmapSource>(icon =>      // вызовется в потоке UI
{
    _icons.Add(icon);
    IconProgress.Value = _icons.Count;
});

try
{
    int n = await _loader.LoadAllSequentialAsync(files, progress, _cts.Token);
}
catch (OperationCanceledException)                      // штатная отмена
{
    Log($"Отменено, успели {_icons.Count}");
}

private void CancelLoad_Click(...) => _cts?.Cancel();     // кнопка «Отмена»
```

КОД · LINQ + ASYNC

Все сразу: Select + Task.WhenAll

```
IEnumerable<Task<BitmapSource>> tasks = files.Select(async file =>
{
    BitmapSource icon = await LoadOneAsync(file, ct);
    progress?.Report(icon);
    return icon;
});

BitmapSource[] all = await Task.WhenAll(tasks);
```

≈ 5 с

Синхронно

окно мёртвое

≈ 5 с

Async по одной

окно живое, прогресс идёт

< 1 с

Async, WhenAll

все ожидания параллельно

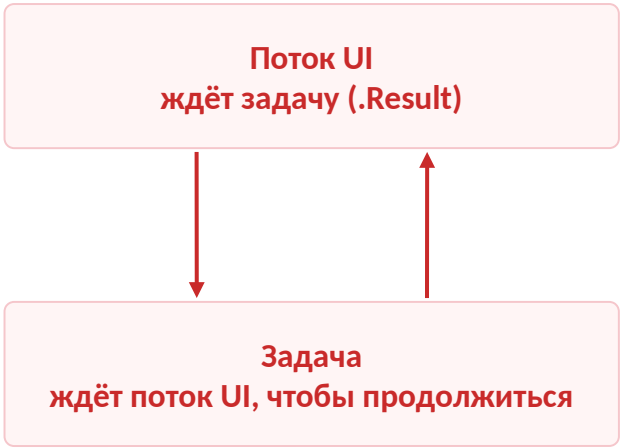
300 файлов, «медленный диск» 15 мс на файл. Async по одной не быстрее — он не замораживает окно.

ГРАБЛИ · ВКЛАДКА 3

.Result: окно ждёт задачу, задача ждёт окно

```
// поток UI встал и ждёт
string text = GetGreetingAsync().Result;

async Task<string> GetGreetingAsync()
{
    await Task.Delay(1000);
    // продолжению нужен поток UI,
    // а он занят ожиданием .Result
    return "Привет";
}
```



Взаимная блокировка навсегда. Правило: в коде с UI никаких .Result и .Wait() — только await.

ГРАБЛИ · ВКЛАДКА 3

Куда деваются исключения

Как вызвали	Что произойдёт с исключением	Итог
`await BrokenAsync()`	Ловится обычным try/catch вокруг await	**правильно**
`async void метод`	catch вокруг вызова не сработает, ошибка роняет приложение	**только для обработчиков**
`BrokenAsync();`	Без await: исключение тихо остаётся внутри Task	**ошибка потеряна**
`await Task.WhenAll(a, b)`	catch получает первое; все — в task.Exception.InnerExceptions	**смотреть все**

Компилятор предупреждает о забытом await (CS4014). Это предупреждение не игнорируйте.

Раунд без таймеров: цикл с await

```
_roundCts = new CancellationTokenSource(TimeSpan.FromSeconds(15)); // сам отменится
try
{
    await Task.WhenAll(SpawnLoopAsync(_roundCts.Token),
                       CountdownAsync(_roundCts.Token));
}
catch (OperationCanceledException) { /* раунд окончен */ }

async Task SpawnLoopAsync(CancellationToken ct)
{
    while (true)
    {
        await Task.Delay(Random.Shared.Next(300, 900), ct);
        Ellipse sphere = SpawnSphere();
        _ = ExpireAsync(sphere, 1500, ct);      // сфера исчезнет сама через 1.5 с
    }
}
```

Async JSON и LINQ вместе

```
await using FileStream fs = File.Create(path);                // ЛР1/ЛР4: сохранение
await JsonSerializer.SerializeAsync(fs, enemies, options);

await using (FileStream input = File.OpenRead(path))        // загрузка
    enemies = await JsonSerializer.DeserializeAsync<List<EnemyTemplate>>(input);

// данные пришли асинхронно — дальше обычный LINQ
var best   = enemies.MaxBy(e => (double)e.BaseGold / e.BaseHealth);
var common = enemies.Where(e => e.SpawnChance / total > 0.15).Select(e => e.Name);
```

async отвечает за то, как дождаться данных. LINQ — за то, что с ними потом сделать.

ИТОГ ЧАСТИ 3–4

async/await: что забираем с собой

1. async — не поток. await освобождает поток на время ожидания. Сколько бы ни ждали, потоков не прибавится.

2. Ждём (файл, сеть, таймер) — await …Async(). Считаем — await Task.Run(…). Свой Thread — редко.

3. Долгая операция = прогресс через IProgress<T> + отмена через CancellationToken.

4. Никаких .Result и .Wait() в UI. async void — только обработчики событий. Не теряйте await.

ШПАРГАЛКА

Одной страницей

LINQ

Where · Select · OrderBy / ThenBy

First(OrDefault) · MaxBy · Count · Sum · Any

GroupBy · OfType<T> · Join · ToDictionary

Запрос ленивый → ToList() фиксирует

Источник меняется в foreach → перебирать копию

async / await

async разрешает await, поток не создаёт

I/O → …Async(), CPU → Task.Run

IProgress<T> + CancellationToken

WhenAll — ждать много задач сразу

Нельзя: .Result, .Wait(), async void, забытый await

К СЛЕДУЮЩЕМУ ЗАНЯТИЮ

Проверить себя

Редактор (ЛР1)

Перевести загрузку иконок на async с прогресс-баром и кнопкой «Отмена». Поиск врага по имени переписать через FirstOrDefault.

Кликер (ЛР2–4)

Нормализацию шансов и выбор врага переписать на LINQ. Сохранение прогресса — на SerializeAsync.

Курсовая

Найти в своём коде все циклы «отбери / отсортируй / посчитай» и заменить на LINQ. Долгие операции — на async.

Код демо: solution piogi3lec3, проекты LinqDemo и AsyncDemo.

Вопросы

LINQ и async/await